Explore | Expand | Enrich

<Codemithra />™

SORT BITONIC DLL

TEST TIME ON LOOP DETECTION

Url:https://forms.gle/M8ByA5m8W4zY47cy5

EXPLANATION

Sorting a biotonic doubly linked list involves arranging the elements in such a way that they first increase in value, then decrease, and so on.

EXPLANATION

1. Identify the Peak Node:

Traverse the doubly linked list from the beginning to find the node where the ascending order ends and the descending order begins. This node is referred to as the "peak node.“

2. Split the List at the Peak Node:

- Once you've located the peak node, split the doubly linked list into two separate lists:

- The first list includes nodes from the start of the original list up to and including the peak node. This part is in ascending order.

- The second list consists of nodes starting from the node immediately after the peak node and continuing to the end of the original list. This part is in descending order.

EXPLANATION

3. Reverse the Second List:

   Reverse the second list (the one in descending order). This will convert it into ascending order.

4. Merge the Two Sorted Lists:

   Finally, merge the two sorted lists (the ascending part and the reversed ascending part) to create a fully sorted biotonic doubly linked list.

The resulting doubly linked list will be sorted in a biotonic manner, with elements first increasing and then decreasing.

SORT BITONIC DLL

EXPLANATION

Consider the random sequence: 1 <-> 5 <-> 8 <-> 12 <-> 11 <-> 7 <-> 6 <-> 2

- Step 1: In the given bitonic doubly linked list (DLL) the peak element is the point where the increasing sequence ends and the decreasing sequence begins. In this case, the peak element is "12."
- step 2:split into two parts first increases (1, 5, 8, 12), then decreases (11, 7, 6, 2).
- Step 3:Separate the list into two parts: the increasing sequence and the decreasing sequence.Increasing Sequence: 1 ,5, 8 , 12 Decreasing Sequence: 11 ,7 , 6, 2

EXPLANATION

Example:

- Step 4:Sort the increasing sequence in second part

  2 <-> 6 <-> 11 <-> 12

- Step 5:Merge the two sorted sequences into a single sorted sequence.

Resultant Sorted Sequence: 1 <-> 2 <-> 5 <-> 6 <-> 7 <-> 8 <-> 11 <-> 12

SORT BITONIC DLL

```
class Main {
//1.node explanation
static class Node {
    int data;
    Node next;
    Node prev;
  }
//2.reverse function
//input 1,2,3---3,2,1
  static Node reverse(Node head_ref) {
    Node temp = null;
    Node current = head_ref;
    while (current != null) {
      temp = current.prev;
      current.prev = current.next;
      current.next = temp;
      current = current.prev;   }
    if (temp != null)
      head_ref = temp.prev;
    return head_ref;
}
//3.Merge operation
//list 1---2,5,8
//list 2---1,3,6
  static Node merge(Node first, Node second)
{
    if (first == null)
      return second;
    if (second == null)
      return first;

    if (first.data < second.data) {
      first.next = merge(first.next,
second);
      if (first.next != null)
        first.next.prev = first;
      first.prev = null;
      return first;
    } else {
```

SORT BITONIC DLL

```
second.next = merge(first, second.next);
      if (second.next != null)
        second.next.prev = second;
      second.prev = null;
      return second;
    }
  }
//4.sorted list----call reverse and merge
function
  static Node sort(Node head) {
    if (head == null || head.next == null)
      return head;
    Node current = head.next;
    while (current != null) {
      if (current.data < current.prev.data)
        break;
      current = current.next;
    }
```

```
if (current == null)
      return head;
    current.prev.next = null;
    current.prev = null;
    current = reverse(current);
    return merge(head, current);
  }
//5.push
  static Node push(Node head_ref, int
new_data) {
    Node new_node = new Node();
    new_node.data = new_data;
    new_node.prev = null;
     new_node.next = (head_ref);
    if ((head_ref) != null)
      (head_ref).prev = new_node;
    (head_ref) = new_node;
    return head_ref;
  }
```

SORT BITONIC DLL

```
//6.print the list
  static void printList(Node head) {
    if (head == null)
      System.out.println("Doubly Linked
list empty");
    while (head != null) {
      System.out.print(head.data + " ");
      head = head.next;
    }
  }
//7.Main function
  public static void main(String args[]) {
    Node head = null;
    // Create the doubly linked list:
    head = push(head, 1);
    head = push(head, 4);
    head = push(head, 6);
    head = push(head, 10);
    head = push(head, 12);
    head = push(head, 7);
    head = push(head, 5);
    head = push(head, 2);
    System.out.println("Original
List:");
    printList(head);
    head = sort(head);
    System.out.println("\nSorted
List:");
    printList(head);
  }
}
```

SORT BITONIC DLL

TIME AND SPACE COMPLEXITY :

Time complexity O(n^2), where 'n' is the number of elements in the linked list. This is because the code uses a nested loop to traverse and sort the list, resulting in quadratic time complexity.

Space complexity O(1) because it uses a constant amount of extra space regardless of the input size.

INTERVIEW QUESTIONS

1.What is a bitonic doubly linked list, and why is it called "bitonic"?

A bitonic doubly linked list is a linked list that exhibits a pattern of monotonically increasing elements followed by monotonically decreasing elements or vice versa. It is called "bitonic" because its behavior resembles a "bitonic sequence" in computer science, which starts with an increasing sequence and then becomes decreasing or vice versa.

2. Explain the process of sorting a bitonic doubly linked list.

To sort a bitonic doubly linked list, you can follow these steps:

1. Find the point where the list transitions from increasing to decreasing (the "bitonic point").
2. Split the list into two sublists: one from the head to the bitonic point and the other from the bitonic point to the end.
3. Reverse the second sublist.
4. Merge the two sublists together.

3.What is the time complexity of sorting a bitonic doubly linked list using the above method?

The time complexity of sorting a bitonic doubly linked list using the above method is O(n log n), where 'n' is the number of elements in the list. The split, reverse, and merge operations each take O(n) time, and there are log(n) levels of recursion in the merge step.

4.How can you optimize the space complexity when sorting a bitonic doubly linked list?

To optimize space complexity, you can use an iterative approach for reversing the second half of the list instead of a recursive one. This way, you can reduce the auxiliary space complexity from O(n) to O(1). Additionally, you can avoid creating new nodes during the merge step by rearranging the pointers of the existing nodes.

https://learn.codemithra.com

Explore | Expand | Enrich

THANK YOU

+91 78150 95095 | codemithra@ethnus.com | www.codemithra.com